

บทที่ 2 เริ่มต้นเขียนไพธอน

2.1 การย่อหน้า (Indentation) ในภาษา Python

การย่อหน้า (Indentation) คือ การเว้นวรรค (Whitespace) หรือการใช้ปุ่ม Tab ที่ต้นบรรทัดของโค้ด เพื่อกำหนดโครงสร้างและขอบเขตของบล็อกคำสั่งต่าง ๆ ในโปรแกรม

ในภาษา Python การย่อหน้ามีความสำคัญอย่างยิ่งต่อไวยากรณ์ (Syntax) ของภาษา ไม่เหมือนกับภาษาโปรแกรมอื่น ๆ ที่มักใช้เครื่องหมายวงเล็บปีกกา ({ }) หรือคำสั่งเฉพาะเพื่อระบุบล็อกของโค้ด

ความสำคัญของการย่อหน้า

1. กำหนดขอบเขตบล็อกคำสั่ง (Code Blocks) การย่อหน้าจะบอก Interpreter ของ Python ว่าคำสั่งใดเป็นส่วนหนึ่งของบล็อกคำสั่งเดียวกัน ตัวอย่างบล็อกคำสั่งที่ต้องใช้การเยื้อง ได้แก่
 - คำสั่งเงื่อนไข (Conditional Statements): if, elif, else
 - คำสั่งวนซ้ำ (Loop Statements): for, while
 - ฟังก์ชัน (Functions): def
 - คลาส (Classes): class
2. โค้ดอ่านง่าย (Readability) การที่ Python บังคับใช้การย่อหน้า ทำให้โค้ดมีโครงสร้างที่ชัดเจนและเป็นระเบียบ ทำให้อ่านและทำความเข้าใจโค้ดได้ง่ายขึ้นทั้งสำหรับผู้เขียนเองและผู้อื่นที่มาอ่านโค้ดนั้น
3. ข้อผิดพลาดจากการย่อหน้า (Indentation Error) หากการเยื้องไม่ถูกต้องตามกฎหมายของ Python (เช่น เยื้องไม่ตรงกัน หรือใช้ Tab และ Space ผสมกันโดยไม่ตั้งค่าให้ถูกต้อง) จะทำให้เกิดข้อผิดพลาด IndentationError และโปรแกรมจะไม่สามารถทำงานได้

รูปแบบการย่อหน้า

ในการย่อหน้าสามารถใช้ได้ทั้งปุ่ม Tab หรือการเว้นวรรค (Spaces) ถ้าใช้การเว้นวรรค นิยมใช้การเว้นวรรค 4 ช่องว่างต่อหนึ่งระดับการย่อหน้า แต่แนะนำให้ใช้ Tab เพราะการเว้นวรรคเอง อาจเกิดข้อผิดพลาดได้ง่ายกว่า และห้ามผสมผสานการใช้ Tab และ Spaces ในไฟล์เดียวกันเด็ดขาด อาจจะทำให้เกิดข้อผิดพลาดได้

ตัวอย่าง

```
score = 85

if score >= 80:
    # บล็อกคำสั่งภายใต้ 'if'
    print("ได้เกรด A")
elif score >= 70:
    # บล็อกคำสั่งภายใต้ 'elif'
    print("ได้เกรด B")
else:
    # บล็อกคำสั่งภายใต้ 'else'
    print("ได้เกรด C")

# คำสั่งนี้ไม่ได้อยู่ภายใต้เงื่อนไขใด ๆ จึงไม่ย่อหน้า
print("สิ้นสุดการตรวจสอบเกรด")
```

2.2 กฎการตั้งชื่อในภาษาไพธอน (Naming Rules)

ในการพัฒนาโปรแกรมด้วยภาษา Python การกำหนด ชื่อ (Identifiers) ให้กับองค์ประกอบต่าง ๆ เช่น ตัวแปร (Variables) ฟังก์ชัน (Functions) คลาส (Classes) หรือ เมธอด (Methods) ถือเป็นสิ่งที่มีความสำคัญอย่างยิ่ง การปฏิบัติตามหลักเกณฑ์การตั้งชื่อที่ภาษา Python กำหนดไว้ จะช่วยเสริมให้โค้ดมีความสามารถในการอ่าน (Readability) ความสอดคล้อง (Consistency) และความง่ายในการบำรุงรักษา (Maintainability)

หลักเกณฑ์พื้นฐานในการตั้งชื่อในภาษาไพธอน

1. การเริ่มต้นของชื่อ

- ชื่อจะต้อง เริ่มต้นด้วยตัวอักษร (A-Z, a-z) หรือ เครื่องหมายขีดล่าง (_) เท่านั้น
- ห้าม เริ่มต้นด้วยตัวเลข
- ตัวอย่างที่ถูกต้อง username, _internal_id, StudentGrade
- ตัวอย่างที่ไม่ถูกต้อง 1stData (เนื่องจากเริ่มต้นด้วยตัวเลข)

2. การคั่นคำในชื่อ

- หากชื่อประกอบด้วยคำมากกว่าหนึ่งคำ ให้คั่นแต่ละคำด้วยเครื่องหมายขีดล่าง (`_`)
- ตัวอย่าง `total_amount`, `first_name`, `calculate_sum`

3. องค์ประกอบของชื่อ

- ชื่อสามารถประกอบด้วย ตัวอักษร (A-Z, a-z), ตัวเลข (0-9) และ เครื่องหมายขีดล่าง (`_`) เท่านั้น
- ห้ามใช้อักขระพิเศษอื่นๆ เช่น `+`, `-`, `*`, `/`, `%`, `@`, `{`, `}`, `:`, `;`, `[`, `]`, `=`, `<`, `>`, `"`, `'`, `\`, ```, `&`, `(`, `)`
- ตัวอย่าง `my_address1`, `maxValue`, `user_account_id`

4. ไม่ซ้ำกับ Keyword ที่สงวนไว้

- ชื่อที่กำหนดขึ้น ต้องไม่ซ้ำกับคำสงวน (Keywords) ที่ภาษา Python กำหนดไว้ ซึ่งเป็นคำที่มีความหมายเฉพาะเจาะจงในไวยากรณ์ของภาษา (เช่น `if`, `for`, `class`, `True`, `False`, `None` เป็นต้น)

5. ความเป็น Case-sensitive

- ภาษา Python เป็นภาษาที่ Case-sensitive ซึ่งหมายความว่าตัวอักษรพิมพ์เล็ก และตัวอักษรพิมพ์ใหญ่มีความแตกต่างกัน
- ดังนั้น `index`, `Index`, และ `inDex` จะถูกพิจารณาว่าเป็นชื่อที่ต่างกัน

6. ความยาวของชื่อ

- ความยาวของชื่อที่กำหนดขึ้น ไม่จำกัดจำนวนตัวอักษร
- อย่างไรก็ตาม ควรพิจารณาความเหมาะสมในการเลือกใช้ชื่อที่มีความกระชับ และสื่อความหมาย

7. การตั้งชื่อที่สื่อความหมาย (Meaningful Naming)

- ชื่อที่กำหนดขึ้น ควรสะท้อนถึงวัตถุประสงค์หรือข้อมูลที่จัดเก็บ เพื่อเพิ่มความชัดเจนและความสามารถในการทำความเข้าใจโค้ด
- ตัวอย่าง: `product_price` ย่อมสื่อความหมายได้ทันทีว่าตัวแปรนี้จัดเก็บค่าราคาสินค้า

2.3 Keyword ในภาษาไพธอน

Keyword หรือ คำสงวน ในภาษาไพธอน คือคำที่ถูกกำหนดไว้สำหรับการใช้งานเฉพาะในภาษา ไม่สามารถนำไปใช้ตั้งชื่อตัวแปร ฟังก์ชัน หรือชื่ออื่น ๆ ได้ เนื่องจากแต่ละคำมีหน้าที่เฉพาะในโครงสร้างของภาษา เช่น การควบคุมเงื่อนไข การวนซ้ำ การนิยามฟังก์ชัน ฯลฯ

ตาราง 3 ตัวอย่าง keyword ที่สำคัญในภาษาไพธอน

and	as	assert	async	await	break
class	continue	def	del	elif	else
except	FALSE	finally	for	from	global
if	import	in	is	lambda	None
nonlocal	not	or	pass	raise	return
TRUE	try	while	with	yield	

หมายเหตุ จำนวนและรายชื่อ keyword อาจแตกต่างกันไปตามเวอร์ชันของไพธอนที่ใช้งาน หากต้องการดู keyword ทั้งหมดในเวอร์ชันที่ใช้งาน สามารถใช้โค้ดนี้ใน Python interpreter

ตัวอย่าง

```
import keyword
print(keyword.kwlist)
```

2.4 คอมเมนต์ (Comments)

คอมเมนต์ คือข้อความที่ถูกเพิ่มเข้าไปในโค้ดเพื่อวัตถุประสงค์ในการอธิบายการทำงานของโค้ดนั้น ๆ ส่วนนี้จะไม่ถูกประมวลผลโดย Interpreter

การใช้คอมเมนต์ช่วยเพิ่มความเข้าใจในโค้ดสำหรับผู้พัฒนาคนอื่น ๆ และสำหรับตัวผู้เขียนเองในอนาคต

- คอมเมนต์แบบบรรทัดเดียว ใช้เครื่องหมาย # นำหน้าข้อความ

ตัวอย่าง

```
# นี่คือนักคอมเมนต์แบบบรรทัดเดียว
print("Hello") # แสดงข้อความ Hello
```

- คอมเมนต์แบบหลายบรรทัด (Docstrings) ใช้เครื่องหมาย `"""` (Triple Double Quotes) คร่อมข้อความ มักใช้เพื่ออธิบายการทำงานของฟังก์ชัน, คลาส, หรือโมดูลทั้งหมด

ตัวอย่าง

```
"""
นี่คือคอมเมนต์หลายบรรทัด
สำหรับอธิบายการทำงานของฟังก์ชันหรือคลาสโดยละเอียด
"""

def greet(name):
    print("สวัสดี", name)
```

2.5 การรับและแสดงผลข้อมูล

การรับและแสดงผลข้อมูลในภาษาไพธอน หมายถึง กระบวนการที่โปรแกรมรับข้อมูลจากผู้ใช้ (Input) และแสดงผลข้อมูลหรือข้อความออกทางหน้าจอ (Output) เพื่อสื่อสารกับผู้ใช้ โดยในภาษาไพธอนมีฟังก์ชันสำคัญที่ใช้ดังนี้

รับข้อมูลด้วยฟังก์ชัน input()

ฟังก์ชัน `input()` ในภาษา Python เป็นฟังก์ชันในตัว (built-in function) ที่มีวัตถุประสงค์หลักในการรับข้อมูลนำเข้า (Input) จากผู้ใช้งานผ่านทางอุปกรณ์นำเข้ามาตรฐาน (Standard Input Device) ซึ่งโดยทั่วไปคือแป้นพิมพ์ (Keyboard) เมื่อมีการเรียกใช้ฟังก์ชันนี้ โปรแกรมจะหยุดการทำงานชั่วคราวและรอให้ผู้ใช้งานป้อนข้อมูลและกดปุ่ม Enter ข้อมูลที่ป้อนเข้ามาจะถูกส่งคืนกลับมาในรูปแบบของ สตริง (String) เสมอ

รูปแบบการใช้งาน (Syntax)

```
input([prompt])
```

พารามิเตอร์ (Parameters)

- `prompt ([ ])` ที่ล้อมรอบพารามิเตอร์ใด ๆ เป็น Optional Parameter มีหรือไม่มีก็ได้
 - เป็นสตริง (string) ที่ใช้แสดงข้อความแจ้งไปยังผู้ใช้งานบนคอนโซล เพื่อบอกให้ทราบว่าควรป้อนข้อมูลใด

- หากไม่ระบุ prompt ฟังก์ชัน input() จะรอรับข้อมูลโดยไม่มีข้อความใด ๆ แสดงขึ้นก่อน

➤ ค่าที่ส่งคืน (Return Value)

- ฟังก์ชัน input() จะส่งคืนค่าข้อมูลที่ผู้ใช้งานป้อนเข้ามาในรูปแบบของ สตริง (String) เสมอ ไม่ว่าผู้ใช้งานจะป้อนตัวเลข ตัวอักษร หรือสัญลักษณ์ใดก็ตาม
- หากผู้ใช้งานไม่ป้อนข้อมูลใด ๆ แล้วกด Enter ฟังก์ชันจะส่งคืนสตริงว่างเปล่า ("")

ตัวอย่าง

```
name = input("Please enter your name: ")
print("Hello, " + name)

age = int(input("Please enter your age: "))
print("You are", age, "years old.")
```

หมายเหตุ การรับข้อมูลที่เป็นตัวเลข ต้องแปลงชนิดข้อมูล เนื่องจาก input() ส่งคืนค่าเป็นสตริงเสมอ หากต้องการนำข้อมูลไปประมวลผลทางคณิตศาสตร์ จะต้องมีการแปลงชนิดข้อมูล (Type Conversion) ก่อน โดยใช้ฟังก์ชันแปลงชนิดข้อมูล เช่น int() สำหรับจำนวนเต็ม หรือ float() สำหรับทศนิยม

แสดงผลข้อมูลด้วยฟังก์ชัน print()

ฟังก์ชัน print() ในภาษา Python เป็นฟังก์ชันในตัว (built-in function) ที่มีวัตถุประสงค์หลักในการแสดงผลลัพธ์ (Output) หรือข้อมูลไปยังอุปกรณ์ส่งออกมาตรฐาน (Standard Output Device) ซึ่งโดยทั่วไปคือหน้าต่างคอนโซล (Console) หรือเทอร์มินัล (Terminal) ที่ผู้ใช้งานมองเห็น

ฟังก์ชันนี้เป็นเครื่องมือพื้นฐานและมีความสำคัญอย่างยิ่งในการพัฒนาโปรแกรม เพื่อใช้ในการตรวจสอบการทำงานของโค้ด แสดงข้อความแจ้งเตือน หรือนำเสนอข้อมูลแก่ผู้ใช้งาน

รูปแบบการใช้งาน (Syntax)

```
print(*objects, sep=' ', end='\n', file=sys.stdout, flush=False)
```

พารามิเตอร์ (Parameters)

➤ *objects (Positional Arguments)

- เป็นวัตถุ (objects) หรือค่า (values) ที่ต้องการแสดงผล สามารถระบุได้มากกว่าหนึ่งค่า โดยคั่นด้วยเครื่องหมายคอมมา (,)
- ฟังก์ชัน print() จะแปลงวัตถุเหล่านั้นให้เป็นสตริง (string) ก่อนที่จะแสดงผล
- * นำหน้าชื่อพารามิเตอร์ (*objects) ในการนิยามฟังก์ชัน นั้นหมายความว่าฟังก์ชันนั้นสามารถรับอาร์กิวเมนต์ (Arguments) แบบ Positional (Positional Arguments) ได้เป็นจำนวนไม่จำกัด

➤ sep (Keyword Argument) (Optional)

- ย่อมาจาก "separator" (ตัวคั่น) เป็นสตริงที่ใช้คั่นระหว่างวัตถุแต่ละชิ้นเมื่อมีการระบุวัตถุมากกว่าหนึ่งรายการ
- ค่าเริ่มต้นคือช่องว่างหนึ่งช่อง (' ')

➤ end (Keyword Argument) (Optional)

- เป็นสตริงที่จะถูกเพิ่มต่อท้ายหลังจากแสดงผลวัตถุทั้งหมดในแต่ละคำสั่ง print()
- ค่าเริ่มต้นคืออักขระขึ้นบรรทัดใหม่ ('\n') ซึ่งทำให้การเรียกใช้ print() แต่ละครั้งขึ้นบรรทัดใหม่เสมอ

➤ file (Keyword Argument) (Optional)

- ระบุวัตถุไฟล์ (file-like object) ที่จะใช้เป็นปลายทางในการเขียนข้อมูล
- ค่าเริ่มต้นคือ sys.stdout ซึ่งหมายถึง Standard Output (คอนโซล)

➤ flush (Keyword Argument) (Optional)

- ค่าเป็นตรรกะ (True หรือ False) ระบุว่า จะบังคับให้บัฟเฟอร์ (buffer) ของสตรีม (stream) ถูกล้างข้อมูลทันทีหรือไม่
- ค่าเริ่มต้นคือ False

ตัวอย่าง การแสดงผลข้อความและตัวเลขพื้นฐาน

```
print("สวัสดีครับ") # แสดงผลข้อความ
print(123)          # แสดงผลตัวเลข
print(3.14159)      # แสดงผลทศนิยม
print(True)         # แสดงผลค่าตรรกะ
```

ตัวอย่าง การแสดงผลวัตถุหลายรายการพร้อมกัน

```
name = "สมชาย"
age = 30
print("ชื่อ:", name, "อายุ:", age, "ปี")
```

ตัวอย่าง การกำหนดตัวคั่น (sep) และตัวจบ (end)

```
print("Apple", "Banana", "Cherry", sep=" - ")
print("บรรทัดแรก", end=" | ")
print("บรรทัดเดียวกัน")
```

2.6 ตัวแปร (Variable)

ตัวแปร (Variable) คือชื่อที่กำหนดขึ้นเพื่อใช้อ้างอิงค่าข้อมูลในหน่วยความจำ ซึ่งค่าข้อมูลนี้สามารถนำไปเรียกใช้ในโปรแกรมได้ การใช้งานตัวแปรช่วยให้โปรแกรมเมอร์สามารถจัดเก็บจัดการ และเข้าถึงข้อมูลได้อย่างสะดวกและมีโครงสร้าง โดยไม่จำเป็นต้องทราบตำแหน่งที่แน่นอนในหน่วยความจำ

ในภาษา Python ตัวแปรมีความยืดหยุ่นสูง เนื่องจาก Python เป็นภาษาแบบ Dynamically-typed ซึ่งหมายความว่าไม่จำเป็นต้องประกาศชนิดข้อมูล (Data Type) ของตัวแปรล่วงหน้า ภาษาจะกำหนดชนิดข้อมูลของตัวแปรโดยอัตโนมัติ ณ เวลาที่โปรแกรมทำงาน (Runtime) ตามค่าข้อมูลที่ถูกกำหนดให้กับตัวแปรนั้น ตัวแปรสามารถอ้างอิงถึงวัตถุที่มีชนิดข้อมูลใดก็ได้ และสามารถเปลี่ยนไปอ้างอิงถึงวัตถุที่มีชนิดข้อมูลอื่นได้ตลอดเวลาในระหว่างการทำงานของโปรแกรม

การประกาศชื่อตัวแปรขึ้นใช้งานจะเป็นไปตามกฎการตั้งชื่อในภาษาไพธอน (Naming Rules) ที่ได้กล่าวก่อนหน้านี้

การกำหนดค่าให้กับตัวแปร (Variable Assignment)

รูปแบบการใช้งาน (Syntax)

variable_name = value

- **variable_name** คือ ชื่อของตัวแปรที่ต้องการกำหนดค่าให้
- **=** คือ ตัวดำเนินการกำหนดค่า (Assignment Operator)

- **value** คือ ค่าข้อมูล หรือนิพจน์ (Expression) ที่จะถูกประเมินผลเพื่อให้ได้ค่า แล้วนำไปกำหนดให้กับตัวแปรนั้น ๆ

ตัวอย่าง

```
x = 10          # กำหนดค่า 10 ให้กับตัวแปร x
name = "Alice"  # กำหนดข้อความ "Alice" ให้กับตัวแปร name
pi = 3.14159    # กำหนดค่าทศนิยมให้กับตัวแปร pi
is_valid = True  # กำหนดค่าบูลีน True ให้กับตัวแปร is_valid
```

การกำหนดค่าหลายตัวแปรพร้อมกัน (Multiple Assignment)

Python อนุญาตให้กำหนดค่าหลายตัวแปรในบรรทัดเดียวได้ ซึ่งเป็นคุณสมบัติที่ช่วยให้โค้ดกระชับ

ตัวอย่าง

```
a = b = c = 1  # กำหนดค่า 1 ค่า ให้กับตัวแปรหลายตัวในบรรทัดเดียว
```

หมายถึง กำหนดให้ตัวแปร a b และ c ซี่อ้างอิงไปยังตำแหน่งบนหน่วยความจำที่เก็บค่าข้อมูลเลขจำนวนเต็ม 10 อยู่

ตัวอย่าง

```
a, b, c = 1, 2, "Hello" # กำหนดค่าหลายตัวแปรในบรรทัดเดียว
```

หมายถึง กำหนดให้ตัวแปร a และ b ซี่อ้างอิงไปยังตำแหน่งบนหน่วยความจำที่เก็บค่าข้อมูลเลขจำนวนเต็ม 1 และ 2 ตามลำดับ และกำหนดให้ตัวแปร c ซี่อ้างอิงไปยังตำแหน่งบนหน่วยความจำที่เก็บค่าข้อมูลสตริง “Hello”

ใช้ฟังก์ชัน input() รับข้อมูลและเก็บลงในตัวแปร

ฟังก์ชัน input() ในภาษาไพธอนใช้สำหรับรับข้อมูลจากผู้ใช้ผ่านคีย์บอร์ด โดยข้อมูลที่รับเข้ามานี้จะถูกเก็บในรูปแบบข้อความ (string) เสมอ การใช้งานจริงจึงมักจะกำหนดตัวแปรเพื่อเก็บข้อมูลที่รับเข้ามา พร้อมทั้งใส่ข้อความแจ้งเตือนเพื่อให้ผู้ใช้ทราบว่าควรป้อนข้อมูลอะไร

รูปแบบการใช้งาน (Syntax)

```
variable_name = input([prompt])
```

ตัวอย่าง

```
first_name = input("Please enter your first name :> ")
last_name = input("Please enter your last name :> ")
```

ใช้ฟังก์ชัน print() แสดงค่าตัวแปรออกทางจอภาพ

ฟังก์ชัน print() ในภาษาไพธอน เป็นฟังก์ชันพื้นฐานที่ใช้สำหรับแสดงค่าตัวแปร ข้อความ หรือ ข้อมูล ต่าง ๆ ออกทางหน้าจอคอนโซล เพื่อให้ผู้ใช้หรือโปรแกรมเมอร์สามารถเห็นผลลัพธ์หรือข้อมูลที่โปรแกรมประมวลผลได้อย่างชัดเจน

รูปแบบการใช้งาน (Syntax)

```
print(variable_name)
```

ตัวอย่าง

```
x = 10
print(x)      # แสดงผลค่าของตัวแปร x
```

ในกรณีที่ต้องการแสดงข้อความร่วมกับตัวแปร

```
print("ข้อความที่ต้องการแสดงผล", variable_name)
```

ตัวอย่าง

```
name = "Alice"
age = 25
print("Name:", name, "Age:", age) # แสดงข้อความร่วมกับตัวแปรหลายค่า
```

การเปลี่ยนค่าของตัวแปร (Reassignment)

การเปลี่ยนค่าของตัวแปร (Reassignment) ในภาษาไพธอน หมายถึง การกำหนดค่าหรือข้อมูลใหม่ให้กับตัวแปรที่มีอยู่แล้ว ซึ่งเป็นการแทนที่ค่าก่อนหน้าด้วยค่าใหม่โดยตรง ภาษาไพธอน อนุญาตให้เปลี่ยนค่าของตัวแปรได้อย่างยืดหยุ่นและไม่จำกัดชนิดข้อมูลเดิมที่เคยเก็บไว้ ตัวอย่างเช่น ตัวแปรที่เคยเก็บข้อมูลชนิดข้อความ (string) สามารถเปลี่ยนเป็นเก็บข้อมูลตัวเลข (int หรือ float) ได้ทันทีโดยไม่ต้องประกาศชนิดข้อมูลใหม่

ตัวแปรสามารถถูกกำหนดค่าใหม่ได้ตลอดเวลา ซึ่งจะทำให้ตัวแปรนั้นไปอ้างอิงถึงวัตถุข้อมูลใหม่ในหน่วยความจำ

ตัวอย่าง

```
x = 10      # กำหนดค่าเริ่มต้นให้ตัวแปร x เป็นจำนวนเต็ม 10
print(x)    # แสดงผล: 10

y = x       # กำหนดค่า y เป็น x
print(x)    # แสดงผล: 3.14

y = "Hello" # เปลี่ยนค่า x เป็นข้อความ "Hello"
print(y)    # แสดงผล: Hello
```

`x = 10` คือ ตัวแปร `x` จะชี้อ้างอิงไปยังตำแหน่งบนหน่วยความจำที่เก็บค่าข้อมูลเลขจำนวนเต็ม 10 อยู่ ดังนั้นตัวแปร `x` ที่ถูกสร้างขึ้นด้วยชนิดข้อมูลเลขจำนวนเต็ม (int)

`y = x` คือ นำค่าของตัวแปร `x` มากำหนดให้ตัวแปร `y` ดังนั้นตัวแปร `y` จะถูกสร้างขึ้นด้วยชนิดข้อมูลเลขจำนวนเต็ม (int) เช่นเดียวกับตัวแปร `x` และชี้อ้างอิงไปยังตำแหน่งบนหน่วยความจำที่เดียวกับตัวแปร `x` ซ้ำอยู่ด้วย

`y = "Hello"` คือ ตัวแปร `y` จะชี้อ้างอิงไปยังตำแหน่งบนหน่วยความจำที่เก็บค่าข้อมูลสตริง "Hello" อยู่ ดังนั้นชนิดข้อมูลของตัวแปร `y` จะถูกเปลี่ยนจากเลขจำนวนเต็ม (int) ไปเป็นสตริง (str)